NAME : ZAID

ROLL : 19

SECTION : A

JVM

The Java Virtual Machine (JVM) is a core component of the Java Runtime Environment (JRE) that allows Java programs to run on any platform without modification. JVM acts as an interpreter between Java bytecode and the underlying hardware, providing Java's famous Write Once, Run Anywhere (WORA) capability.

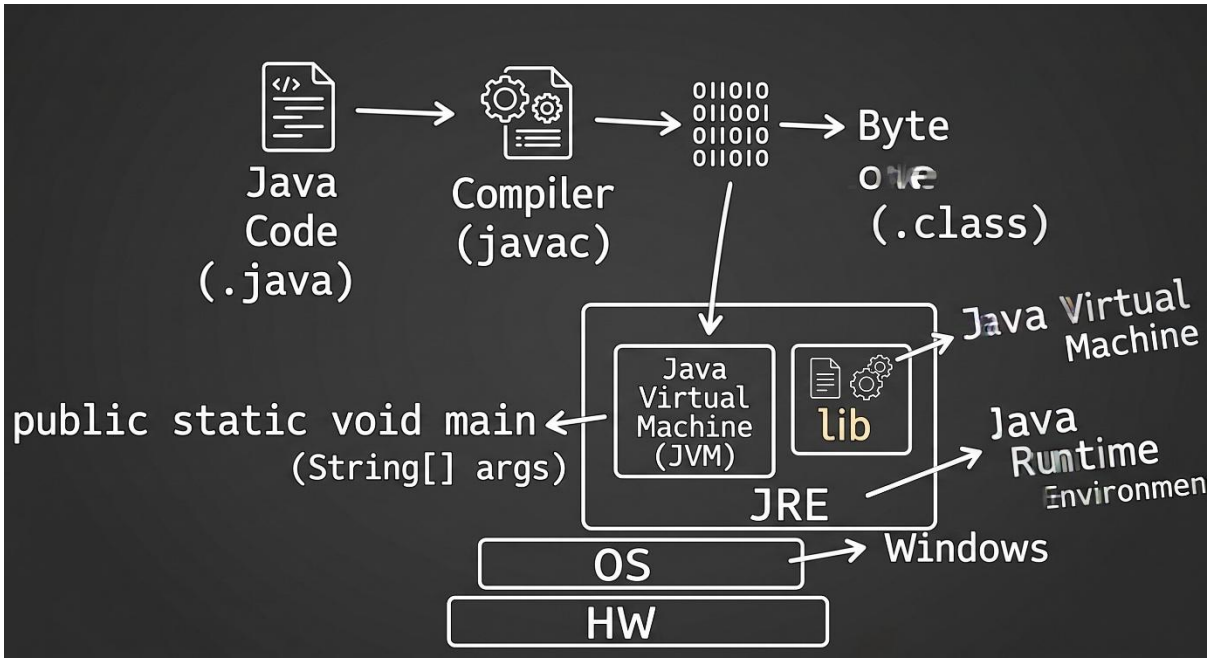
How the JVM Works

- Java source (.java) -> compiled by javac -> bytecode (.class)
- JVM loads the bytecode, verifies it, links it, and then executes it
- Execution may involve interpreting bytecode or using Just-In-Time (JIT) compilation to convert "hot" code into native machine code for performance
- Garbage collection runs in the background to reclaim memory from unused objects

Core Functions

- **Portability:** Translates universal bytecode into hardware-specific machine code.
- **Security:** Verifies bytecode instructions before execution to prevent malicious operations.
- **Multi-language Support:** Runs any language that compiles into standard Java bytecode.

Visual Representation of JVM

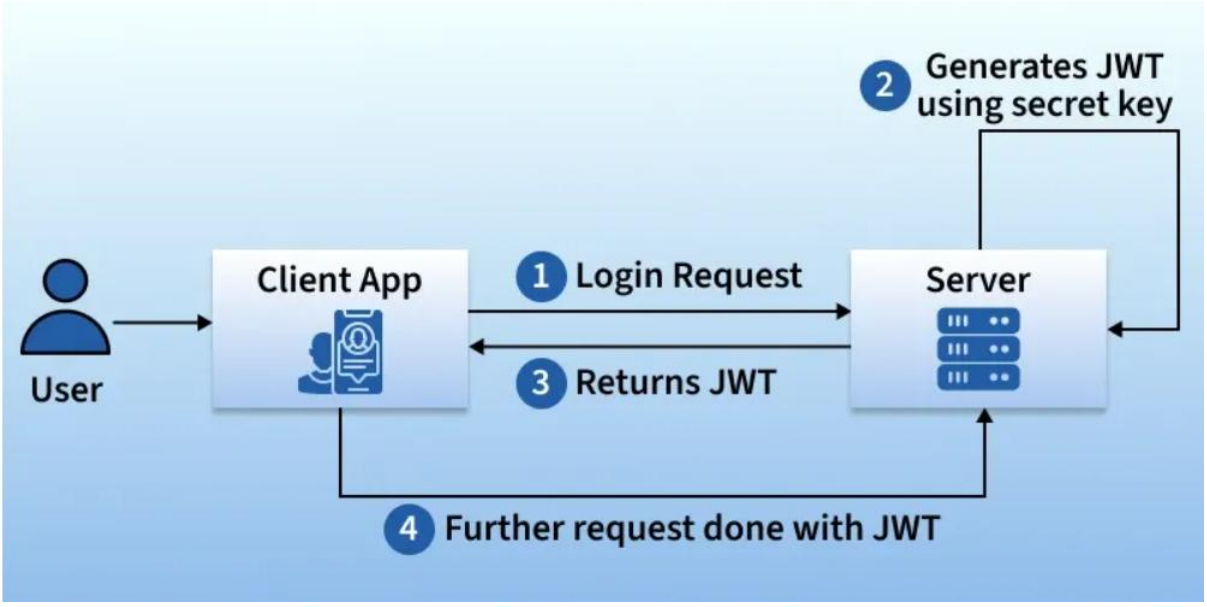


JSON Web Token (JWT) is a compact and secure method for transmitting information between parties as a JSON object. It is commonly used for authentication and authorization in web applications, allowing users to access protected resources without repeatedly providing credentials. Key features include:

- **Stateless Authentication: Stores user information in a token, reducing the need for server-side session storage.**
- **Secure Data Exchange: Uses digital signatures to verify that the token has not been altered.**
- **Compact Format: Consists of three parts—Header, Payload, and Signature—encoded into a single string.**
- **Widely Supported: Works across different programming languages and platforms.**

JWT Authentication Working

JWT is used for authentication. This is the whole flow of the Authentication process:



Here are the key steps:

- **Login Request: The user logs in through the client application (e.g., web or mobile app) by sending their credentials (username & password) to the server.**
- **Server Generates JWT: If the credentials are correct, the server generates a JWT token using a secret key.**
- **Returns JWT: The server sends the JWT back to the client application.**
- **Further Requests with JWT: For any subsequent requests, the client sends the JWT along with the request. The server verifies the JWT before granting access to protected resources.**

Functional Programming Paradigm

Lambda calculus, developed by Alonzo Church, is a minimal framework for studying computation through functions. It defines what is computable and is equivalent to a Turing machine in power. It offers a theoretical model for describing functions and their evaluation, and forms the foundation of most modern functional programming languages.

Fact: Alan Turing was a student of Alonzo Church who created Turing machine which laid the foundation of imperative programming style.

Programming Languages that support functional programming: Haskell, JavaScript, Python, Scala, Erlang, Lisp, ML, Clojure, OCaml, Common Lisp, Racket.

Concepts of Functional Programming

- **Pure Function**
- **Recursion**
- **Referential transparency**
- **Functions are First-Class and can be Higher-Order**
- **Variables are Immutable**

Pure Functions

These functions have two main properties.

- **First, they always produce the same output for same arguments irrespective of anything else.**
- **Secondly, they have no side-effects i.e. they do not modify any arguments or local/global variables or input/output streams. Later property is called immutability. The pure function's only result is the value it returns. They are deterministic.**

Programs done using functional programming are easy to debug because pure functions have no side effects or hidden I/O. Pure functions also make it easier to write parallel/concurrent applications. When the code is written in this style, a smart compiler can do many things - it can parallelize the instructions, wait to evaluate results when needing them, and memorize the results since the results never change as long as the input doesn't change.

Example of the Pure Function

```
sum(x, y)         // sum is function taking x and y as arguments
  return x + y    // sum is returning sum of x and y without changing them
```

Recursion

There are no “for” or “while” loop in functional languages. Iteration in functional languages is implemented through recursion. Recursive functions repeatedly call themselves, until it reaches the base case.

Example of the recursive function:

```
fib(n)
  if (n <= 1)
```

```
    return 1;
  else
    return fib(n - 1) + fib(n - 2);
```

Referential Transparency

In functional programs variables once defined do not change their value throughout the program. Functional programs do not have assignment statements. If we have to store some value, we define new variables instead. This eliminates any chances of side effects because any variable can be replaced with its actual value at any point of execution. State of any variable is constant at any instant.

Example:

```
x = x + 1 // this changes the value assigned to the variable x.
      // So the expression is not referentially transparent.
```

Functions are First-Class and can be Higher-Order

First-class functions are treated as first-class variable. The first class variables can be passed to functions as parameter, can be returned from functions or stored in data Structures. Higher order functions are the functions that take other functions as arguments and they can also return functions.

Example:

```
show_output(f)        // function show_output is declared taking argument f
                  // which are another function
  f();             // calling passed function


print_gfg()         // declaring another function
  print("hello gfg");


show_output(print_gfg)  // passing function in another function
```

Variables are Immutable

In functional programming, we can't modify a variable after it's been initialized. We can create new variables – but we can't modify existing variables, and this really helps to maintain state throughout the runtime of a program. Once we create a

variable and set its value, we can have full confidence knowing that the value of that variable will never change.

Advantages of Functional Programming

- **Pure functions are easier to understand because they don't change any states and depend only on the input given to them. Whatever output they produce is the return value they give. Their function signature gives all the information about them i.e. their return type and their arguments.**
- **The ability of functional programming languages to treat functions as values and pass them to functions as parameters make the code more readable and easily understandable.**
- **Testing and debugging is easier. Since pure functions take only arguments and produce output, they don't produce any changes don't take input or produce some hidden output. They use immutable values, so it becomes easier to check some problems in programs written uses pure functions.**

Disadvantages of Functional Programming

- **Sometimes writing pure functions can reduce the readability of code.**
- **Writing programs in recursive style instead of using loops can be bit intimidating.**
- **Writing pure functions are easy but combining them with the rest of the application and I/O operations is a difficult task.**
- **Immutable values and recursion can lead to decrease in performance.**

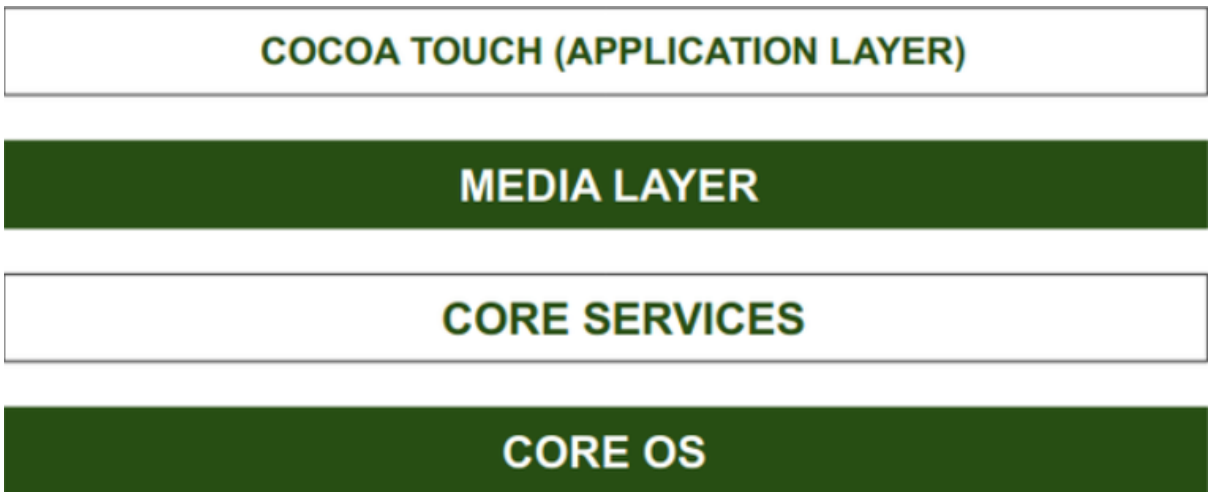
Architecture of IOS Operating System

IOS is a mobile operating system that Apple Inc. has designed for its iPhones, iPads, and Apple mobile devices. IOS is a mobile operating system and is the second most popular and widely used after Android.

The structure of the iOS operating system is Layered based. Its communication doesn't occur directly. The layers between the Application Layer and the Hardware layer will help with Communication. The lower level gives basic services on which all applications rely and the higher-level layers provide graphics and interface-

related services. Most of the system interfaces come with a special package called a framework.

A framework is a directory containing dynamic shared libraries, such as .files, header files, images, and helper applications that support the library. Every layer has its associated frameworks useful for a developer.



Core OS Layer

All the IOS technologies are built under the lowest level layer i.e. Core OS layer. These technologies include:

Core Sercices Layer

Some important frameworks are present in the CORE SERVICES Layer which helps the iOS operating system to cure itself and provide better functionality. Architecture as

Media Layer

By taking the media layer's help, we will enable all graphics video, and audio technology of the system. This is the second layer in the architecture.

Cocoa Touch

COCOA Touch is also known as the application layer which acts as an interface for the user to work with the iOS Operating system. It supports touch and motion events and many more features.

Advantages of IOS Operating System

The iOS operating system has some advantages over other operating systems available in the market especially the Android operating system. Here are some of them-

- **More secure than other operating systems**
- **Fluid responsive with a great UI**
- **Most Suitable for Business and Professionals**
- **Produce less heat compared to Android**

Disadvantages of IOS Operating System

Let us have a look at some disadvantages of the iOS operating system-

- **More Expensive.**
- **Less User Friendly than the Android Operating System.**
- **Not Flexible remain to support only IOS devices.**
- **Battery Performance Decreases.**

Android Architecture

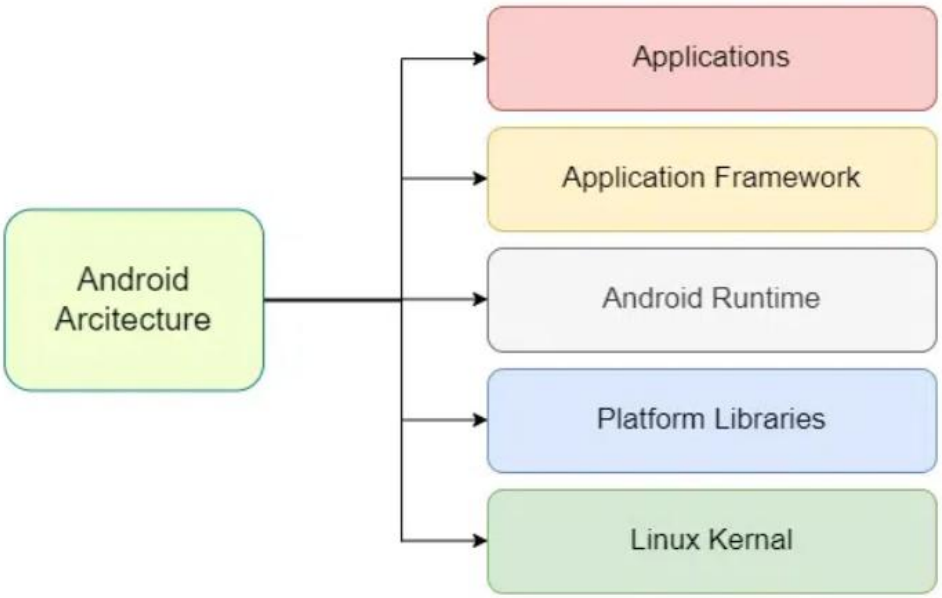
Android architecture contains a different number of components to support any Android device's needs. Android software contains an open-source Linux Kernel having a collection of a number of C/C++ libraries which are exposed through application framework services. Among all the components Linux Kernel provides the main functionality of operating system functions to smartphones and Dalvik Virtual Machine (DVM) provide a platform for running an Android application.

Components of Android Architecture

The main components of Android architecture are the following:-

- **Applications**
- **Application Framework**
- **Android Runtime**
- **Platform Libraries**
- **Linux Kernel**

Pictorial representation of Android architecture



1. Applications

Applications is the top layer of android architecture. The pre-installed applications like home, contacts, camera, gallery etc and third party applications downloaded from the play store like chat applications, games etc. will be installed on this layer only. It runs within the Android run time with the help of the classes and services provided by the application framework.

2. Application framework

The Application Framework is a core part of Android that gives developers the tools and services they need to build apps. It provides access to device features like hardware, screen display, and system resources. It includes several important services that make it easier to build powerful and consistent Android apps without having to create everything from scratch.

3. Application runtime

4. Platform libraries

The Platform Libraries includes various C/C++ core libraries and Java based libraries such as Media, Graphics, Surface Manager, OpenGL etc. to provide a support for android development.

Example : Media,Surface Manager

5. Linux Kernel

Linux Kernel is heart of the android architecture. It manages all the available drivers such as display drivers, camera drivers, Bluetooth drivers, audio drivers, memory drivers, etc. which are required during the runtime. The Linux Kernel will provide an abstraction layer between the device hardware and the other components of android architecture. It is responsible for management of memory, power, devices etc. The features of Linux kernel are:

- **Security: The Linux kernel handles the security between the application and the system.**
- **Memory Management: It efficiently handles the memory management thereby providing the freedom to develop our apps.**
- **Process Management: It manages the process well, allocates resources to processes whenever they need them.**
- **Network Stack: It effectively handles the network communication.**
- **Driver Model: It ensures that the application works properly on the device and hardware manufacturers responsible for building their drivers into the Linux build.**